Analysis of Sparsely Populated Tables (1-2 Rows)

You noticed that several tables in your active Supabase database only have **1 or 2 rows of data**.

Why is this happening? When an application is still in early development, testing, or recently launched, seeing tables with 1 or 2 rows is incredibly common.

Here is the full validation mapping of exactly **why** specific tables only hold 1 or 2 rows right now, and whether that is expected behavior or a bug.

Category 1: "The Initialization Seed" (Expected: 1 Row)

These tables represent the very foundation of your app. They often only have 1 row because you are currently testing/running the app within a single overarching environment or company account.

- `orgs` / `organizations`: You likely only have **1 row** here representing the single tenant company ("Ottobon") you are using to test the app.
- `company_details`: Same reason. Only one company is configured right now.

Validation: **100% Valid and Expected.** These will slowly grow as you onboard entirely new client organizations.

Category 2: "The Admin Test User" (Expected: 1-2 Rows)

These tables represent the user accounts. If only you (and maybe one developer) have logged into the system to test it, these tables will only have 1 or 2 rows.

- `profiles`: The users who have signed up.
- `departments`: E.g., You might have just created an "Engineering" department to test profile assignments.
- `attendance`, `employee_finance`, `payroll`: If you have only tested clocking in or generating a payslip for *your own* admin test user once or twice, these will only hold a single row.

Validation: **100% Valid.** These will organically grow as more employees create accounts and use the app.

Category 3: "The Proof of Concept Feature" (Expected: 1-2 Rows)

These are features that you built and tested exactly *once* to make sure they worked, but haven't actively used since.

- `projects`: You created one test project (e.g., "Alpha Launch") to see if the UI works.
- `tasks`: You created one or two test tasks inside that test project.
- `task_steps`: You added a checklist to that single test task.
- `leaves`: You submitted one test "Sick Day" request to ensure the AI analysis trigger fired successfully.
- `leave_ai_analysis`: Holding the single AI response from that one test leave request.
- `skills_master`: You added "React" and "Node.js" just to verify the dropdown worked.

Validation: **Valid, but Temporary.** These 1-2 rows are essentially "dummy data." You can freely delete these rows to wipe the slate clean before a real production launch without hurting the database.

Category 4: "Isolated Interactions" (Expected: 1-2 Rows, Warning if stuck)

These tables represent interactions between entities.

- `conversations` / `conversation_members`: You started a single test chat with yourself.
- `messages` / `attachments`: You sent "Hello World" or uploaded one test image to verify the storage bucket policy worked.
- `project_members`: You assigned your admin profile to the single test project.

Validation: **Valid Testing Remnants.**

🛠️ Validation Checklist & Next Steps

Are 1-2 rows a bad thing? No. It simply proves the database is receiving successful `INSERT` commands from your front-end or API, but hasn't reached a scale of heavy daily usage yet.

What should you do?

1. **If you are preparing for a real Launch:** You should write a TRUNCATE script to wipe the dummy data out of Category 3 and 4 tables so the app starts "fresh" for real users.
2. **If you are still Actively Developing:** Leave the 1-2 rows there! They act as perfect mock data to ensure your UI tables, queries, and `SELECT` statements don't throw empty-array errors while you code.

🚨 Can I just delete these tables?

NO, you cannot delete the tables that have 1 or 2 rows.

Those sparsely populated tables are your **Core Essential Tables**. They only have 1 or 2 rows right now because you are currently the only person testing the app.

For example, tables like:

- `orgs` (1 row because you only have one test company)
- `profiles` (1 or 2 rows because only you and maybe a test user have logged in)
- `projects` & `tasks` (1 or 2 rows because you only created a couple of items to test the UI)
- `attendance` & `leaves` (1 or 2 rows because you only tested the feature once)

If you delete these tables, your entire application will break. The front-end code expects these tables to exist so users can log in, view projects, and create tasks.

Summary: What to Delete vs. What to Keep

1. **Keep the tables with 1-2 rows:** These are your required core tables holding your initial test data.
2. **Delete the completely empty tables:** Those are the unused modules (Candidates, Invoicing, redundant audit logs) that you aren't actually using right now.